AI-Powered Campus Placement Portal

with Explainable Resume Matching, Skill-Gap Analysis & Career Readiness

Semester VII | Team of 6 | Software-only project | Proposed continuation into Semester VIII

1. Project Overview

Item	Details
Proposed topic	AI-Powered Campus Placement Portal with Explainable Resume Matching, Skill-Gap Analysis and Career Readiness
Nature of project	Full-stack web application with a separate AI/ML service
Problem source	Self-identified problem: fragmented campus placement workflows and limited personalized feedback for students
Team	6 members: 5 full-stack developers and 1 data science specialist
Project duration	Semester VII (16-week delivery plan), with optional advanced modules in Semester VIII subject to department approval
Project guide	[Guide name and designation]
Department / Institute	[Department, College name]

2. Brief Description

The proposed platform connects students, recruiters and the placement cell through a single, secure web application. It supports resume parsing, explainable resume-to-job matching, job recommendations, application tracking, interview scheduling, skill-gap analysis and personalized learning plans. Recruiters can publish jobs and review evidence-backed candidate match explanations, while placement administrators can manage drives and view placement analytics. A readiness simulator with assessments and progress tracking extends the system beyond recruitment administration. The solution uses React, Spring Boot, PostgreSQL and a Python/FastAPI ML service.

3. Problem Statement

Campus placement activities are often distributed across spreadsheets, email threads and messaging groups. Students can miss suitable openings and receive little actionable feedback about why their profiles may not meet a role's requirements. Recruiters spend time screening resumes manually, while placement cells may lack a reliable, consolidated view of applications, shortlists, offers and batch-level skill needs. The proposed system centralizes placement workflows and provides explainable, measurable decision support. AI-generated scores will support—not replace—human review.

4. Objectives

- Provide secure, role-based portals for students, recruiters and placement administrators.
- Parse resumes into structured profile information and match candidates to jobs using transparent, explainable criteria.
- Identify role-specific skill gaps and recommend learning resources and practical projects.
- Manage job posts, applications, shortlists, interview schedules, feedback and notifications.
- Provide placement analytics for application funnels, branch/role trends, offers and common skill gaps.
- Measure the performance of resume parsing and candidate ranking using defined evaluation metrics.
- Add a placement-readiness simulator with assessments, progress tracking and personalized improvement plans.
- Deliver a tested, containerized, documented application with privacy, security and deployment practices.

5. Scope and Feature Modules

Module	Features
Student portal	Registration and profile; resume upload and versioning; extracted skills/education/experience review and correction; job recommendations; explainable match score; eligibility checklist; one-click application; application status timeline; skill-gap report; learning roadmap; email/in-app notifications; readiness assessment history.
Recruiter portal	Company profile; company verification workflow; job creation and eligibility criteria; candidate search and AI-assisted ranking; matched/missing-skill evidence; shortlist/reject/hold actions with reason codes; interview scheduling; feedback entry; application pipeline management.
Placement-cell / admin portal	Approve companies and job posts; manage students and recruiters; bulk student import with validation; placement drives; branch/role-wise dashboards; application-to-shortlist conversion; offer and package trends; common skill-gap analytics; exportable reports; audit logs.
Resume intelligence service	PDF/DOCX text extraction; parsing of skills, education, projects and experience; normalization of skill synonyms; confidence indicators; student correction workflow; malformed-file handling; no unsupported claims when information is missing.
Explainable matching engine	Eligibility rules; weighted skills and requirements; semantic similarity using text embeddings; configurable weights; match breakdown; matched and missing requirements; resume evidence snippets; comparison of baseline keyword matching with semantic/hybrid ranking.
Skill-gap and learning recommender	Compare verified profile skills with target-role requirements; prioritize gaps by importance; recommend curated courses, documentation and project exercises; create a learning roadmap; track completion and reassess progress.
Placement readiness simulator	Target-role selection; technical MCQs and coding-assessment links or in-platform questions; optional structured mock-interview module; topic-wise results; strengths and weak areas; readiness checklist; improvement plan and progress over time. Treat readiness as an estimate, not a placement guarantee.
Scheduling and notifications	Interview calendar; schedule/reschedule/cancel workflow; reminders; email and in-app notifications; application and shortlist status alerts; notification preferences.
Analytics and reporting	Placement funnel; drive-level outcomes; branch and role breakdowns; offer/package distributions where data is available; popular missing skills; recruiter activity; model evaluation and service performance metrics; CSV export.
Security, privacy and audit	Role-based access control; secure password hashing; validated uploads; consent and purpose notice; controlled resume access; data retention/deletion workflow; audit trail for key actions; secrets management; anonymized or synthetic data for testing.

6. AI/ML Design and Evaluation

6.1 Proposed matching approach

Use a hybrid pipeline: (1) hard eligibility checks such as graduation year or required criteria, (2) structured skill/requirement matching, (3) semantic similarity between job descriptions and resume content, and (4) a configurable ranking function. Display a breakdown of the score and evidence behind each component. The initial weights should be treated as a baseline and calibrated using a documented evaluation set—not presented as objectively correct without validation.

6.2 Evaluation plan

- Resume extraction: precision, recall and F1 for fields/skills against manually reviewed sample annotations.
- Candidate ranking: Precision@K and Recall@K against a documented relevance-labelled test set.
- Baseline comparison: compare keyword-based ranking with semantic and hybrid approaches.
- Operational measures: inference latency, failure rate and successful processing rate.

- Human review: collect structured feedback from team members or consenting pilot users; document limitations and possible bias.

The project will use public, synthetic or authorized anonymized samples during development. AI rankings must be reviewable by humans, and protected or irrelevant personal attributes should not be used as ranking signals. Course recommendations should come from a curated catalogue or verified links.

7. System Architecture and Technology Stack

Layer	Suggested technology	Responsibility
Frontend	React, Tailwind CSS	Student, recruiter and admin interfaces; dashboards; forms; accessible responsive UI.
Backend API	Java, Spring Boot, Spring Security, JWT or secure session approach	Business logic, authorization, user/job/application workflows, validation and notifications.
ML service	Python, FastAPI, spaCy, sentence-transformers	Resume parsing, skill normalization, semantic matching, skill-gap analysis and recommendation endpoints.
Database	PostgreSQL	Users, roles, student profiles, jobs, applications, interviews, assessment results, audit events and model evaluation records.
File storage	Local storage for development; object storage for deployment	Resume storage with access controls, file validation and retention policies.
Testing	JUnit, Spring Boot Test, pytest, Postman, frontend test framework	Unit, API, integration, ML evaluation and end-to-end tests.
DevOps	Docker, Docker Compose, GitHub Actions	Repeatable local environment, CI checks, containerized services and deployment pipeline.
Documentation	OpenAPI/Swagger, README, architecture diagrams	API contracts, setup instructions, deployment guide and technical documentation.

7.1 Core data entities

User, Role, StudentProfile, RecruiterProfile, Company, Resume, ExtractedSkill, Job, JobRequirement, JobMatch, Application, Interview, Feedback, CourseResource, SkillGapReport, LearningPlan, Assessment, AssessmentAttempt, Notification, PlacementDrive and AuditEvent. Final schema and relationships will be confirmed during requirements and design.

7.2 Suggested service flow

React client → Spring Boot REST API → PostgreSQL / controlled file storage. The backend invokes the FastAPI service for resume parsing, matching, skill-gap analysis and model evaluation. Define and version the API contract early, use mock responses during parallel development, and integrate continuously.

8. Team Roles and Responsibilities

# / Role	Ownership
1. Backend lead	Authentication, authorization, user management, API conventions, code review and integration oversight.
2. Backend developer	Job/application workflows, interview scheduling, notifications and related business rules.
3. Frontend developer	Student portal, resume upload, job recommendations, match explanation and skill-gap views.
4. Frontend developer	Recruiter portal, placement-cell dashboard, charts and reporting UI.
5. DevOps and QA	Database design support, Docker, CI/CD, test strategy, integration testing, deployment and documentation.
6. Data science specialist	Resume parser, matching/ranking pipeline, skill-gap engine, evaluation dataset and ML API.

Team member names and roll numbers: [Member 1] through [Member 6].

9. Implementation Plan (16 Weeks)

Weeks	Activities	Deliverable
1–2	Finalize requirements, scope, roles, user stories and acceptance criteria; identify authorized sample data; establish repository and task board.	Approved scope, backlog and project plan.
3–4	Architecture and database design; API contracts; wireframes; threat/privacy review; CI setup; ML baseline and mock API.	SRS, schema, architecture diagram, API specification.
5–7	Build authentication, role permissions, student/recruiter profiles, jobs, resume upload, applications and basic UIs.	End-to-end core workflow.
8–9	Implement resume extraction, skill normalization, baseline matching and initial skill-gap reports; add unit tests.	Working MVP with baseline AI.
10–12	Add explainable match breakdown, learning recommendations, scheduling, notifications, admin dashboards and first model evaluation.	Integrated feature-complete beta.
13–14	Security and privacy checks, integration/end-to-end tests, performance checks, Docker deployment and bug fixing.	Deployed candidate build and test report.
15–16	Pilot/demo rehearsal, user guide, API documentation, final report, presentation and limitations/future-work review.	Final demo, report and documented release.

Semester VIII extension, subject to approval: placement-readiness simulator enhancements, coding assessments, structured mock interviews, larger evaluation set, recruiter feedback analytics, accessibility improvements and performance tuning.

10. Testing, Quality and Acceptance Criteria

- All major user flows work end to end for student, recruiter and admin roles.
- Unauthorized users cannot access restricted APIs or resumes; access checks are tested server-side.
- Invalid, oversized or unsupported resume files are rejected safely and produce understandable errors.
- A reviewer can inspect why a match score was produced and correct inaccurate extracted profile fields.
- Matching and extraction metrics are reported on a documented test set, including limitations and examples of failure.
- Database migrations, automated tests and service startup are reproducible from documented instructions.
- A clean environment can run the application using the documented Docker setup, subject to configured dependencies.
- No real student resume or secret credential is committed to the repository.

11. Expected Outcomes

- A working web platform for students, recruiters and placement administrators.
- An ML service for resume parsing, explainable job matching, skill-gap analysis and learning recommendations.
- A readiness assessment and progress-tracking foundation for students preparing for target roles.
- A placement dashboard that summarizes applications, shortlists, offers and aggregate skill needs.
- Documented evaluation results for resume extraction and candidate ranking.
- A secure, tested and containerized deployment with API documentation, test report, README and final capstone report.

12. Risks and Mitigation

Risk	Mitigation
ML service delays integration	Agree on API schemas by week 4; use mock responses; build a deterministic baseline before advanced semantic features.
Scope creep	Freeze P0 scope by week 2; treat readiness simulator enhancements and advanced analytics as optional until core workflows are stable.

Poor or limited training/evaluation data	Use synthetic/public/authorized data; manually label a small evaluation set; report dataset limitations transparently.
Biased or misleading rankings	Use job-related signals, show explanations, keep human review, evaluate failure cases and avoid claims that scores guarantee hiring.
Resume privacy and security	Obtain appropriate consent, restrict access, validate files, define retention/deletion, avoid public exposure and use anonymized test data.
Late integration and deployment problems	Integrate weekly, run CI checks, use Docker Compose and deploy a minimal version early.
Unverified course recommendations	Maintain a curated catalogue of resources and label recommendation relevance; do not fabricate course availability or credentials.

13. Ethical, Privacy and Responsible-AI Considerations

- The system is decision support, not an automated final hiring authority.
- Students should be able to review and correct extracted resume data.
- Only relevant, job-related criteria should affect matching; protected attributes should not be used as ranking features.
- Provide clear notice about processing, access controls, retention and deletion.
- Show uncertainty and limitations when parsing is incomplete or evidence is insufficient.
- Keep audit records for significant administrative and recruitment workflow actions.

14. Sustainable Development Goals

SDG	Contribution
SDG 4: Quality Education	Personalized skill-gap reports, curated learning resources and role-based learning plans.
SDG 8: Decent Work and Economic Growth	More transparent job discovery, application workflows and feedback to support employability.
SDG 9: Industry, Innovation and Infrastructure	A modular digital platform and applied AI service that can be adapted by institutions.

15. Final Scope Recommendation

The minimum viable release should prioritize secure role-based access, job/application workflows, resume parsing, explainable matching, skill-gap reports, basic placement analytics, evaluation metrics and deployment. The readiness simulator, advanced mock interviews and expanded analytics should be delivered only after the core product is stable. This scope balances a meaningful AI contribution with the delivery capacity of a six-person team.

16. Approval

Prepared by	Project Guide	Department Approval
Names / Roll Numbers: _____ Date: _____	Name / Signature: _____ Date: _____	Signature / Date: _____